

Object-Oriented Programming (OOP, CCIT4023)

HKU SPACE Community College, 2022-2023

Assignment 2

(15%)

(Total Marks: 30)

- Finish this work, based on concepts and techniques learnt in our course.
- Students should finish reviewing the related course notes and materials, before doing this assignment.
- Individual work: Student MUST FINISH THIS WORK ALONE. Student cannot work with others.
* Plagiarism / Collusion / Shared work with others are not allowed. Zero mark will be given, with possible disciplinary action.
- Questions related to program codes are based on Java programming language, unless specified.
- Follow given instructions and guidelines.

Section A, A2A (10 marks): ONLINE (SOUL-Quiz Feature)

Multiple Choice (MC) and Matching Questions

- Identify and select the option of choice that "best" completes the statement, matches the item, or answers the question.
- Number of Attempts Allowed: 2
- Grading method: Highest Grade
- Make sure you have *successfully* completed, "Finished" and submitted your work.
* Attempts must be submitted before time expires, or they are NOT counted.
- Questions related to program codes are based on Java programming language, unless specified.
- 10 MC questions and 10 Matching questions. Each question carries the same mark.

Section B, A2B (20 marks): Programming Tasks

* Students may reference to the Practicing Labs to complete this task. Refer to the lab documents for details if necessary.

General requirements (unless further specified):

- Unless specified, *no data validation required* in the program (program runs in normal situations).
- Unless specified, do not create your own package.
- Unless specified, all related files (including *.java, *.class) should be working *in the same folder*.
- Each source code *.java file must *start with comments*, including the information of individual student (**student name, student id, class number, year of study**) as the given file, unless specified.

Given Materials:

- This assignment document.
- The Java files **StudentInfo.java** and **BkOrderSys.java** are to be modified and completed by student. Also modify the top comments for your own **Student Information**.
- Java file **MainTA2B.java**, Main Testing class for testing purpose only. No need to submit.
 - DO NOT modify the given Main Testing file.

A2B (20 marks)**Develop a Java Program to Model a *Book Order System***

Write a Java program of handling purchase orders by fulfilling the following requirements, which includes three classes: **StudentInfo**, **BookOrder**, and **BkOrderSys**.

** This assessment work may not reflect a real and complete scenario in actual cases.*

1. Class **StudentInfo [8 marks]**

- Create a public Java class **StudentInfo**, in file **StudentInfo.java**, to model student (for data records) based on the given UML, the naming and the descriptions below.
- The class **StudentInfo** have four fields, representing data information, as the field names and types in the UML class diagram.
 - 1) Student ID 2) Name
 - 3) Phone number 4) Part-time student or not
- One public constructor which accepts 4 input parameters which simply sets its fields with these input parameters accordingly.
- A related method returns `true` if the phone number is valid, returns `false` otherwise.
 - Suppose a valid phone should be an integer within (inclusive) 20000000 and 99999999
- A related method returns student info as a string line in the form below (and later sample display):

`"<sID>,<sName>,<sPhone>,<sPT??>"`

 - E.g. for a part-time student `"20210001,CHAN Tai Man,98765432,PartTime"`
 - E.g. for a full-time student `"20210002,CHAN Siu Ming,98769876"`

StudentInfo
sID: int sName: String sPhone: int sPT: boolean
+ StudentInfo(id:int, name:String, phone:int, pt: boolean) + isValidPhone(): boolean + getSInfo(): String

2. Class **BookOrder [6 marks]**

- Create a public Java class **BookOrder**, in file **BookOrder.java**, to model a book order (for data records) based on the naming and the descriptions below.
- The class has 4 fields representing the book order ID, description, and student:
 - Name `bID` of type `int` in Java, representing the ID of the order
 - Name `bTitle` of type `String` in Java, representing the title of the book
 - Name `bPrice` of type `double`, representing the price (HKD) of this order
 - Name `bStudent` of type `StudentInfo`, representing the student who places this order
- The class has one public constructor which accepts 4 input parameters. The 4 parameters are in order as (id, title, price, student). The constructor simply sets its fields with these input parameters accordingly.
- The class has one method, named `dispBOInfo()` which accepts no parameter and no return value. This method displays the information of this book order on console in multi-lines in the format below (and the later sample display):

```
Order ID: <bID>
Title: <bTitle>
Price (HKD): <bPrice>
Student: <sID>,<sName>,<sPhone>,<sPT??>
```

Sample Program Outputs by Calling the Methods dispBOInfo() :

Order ID: 873

Title: Seven Little Plays

Price(HKD): 225.0

Student: 20210001, CHAN Tai Man, 98765432, PartTime

Order ID: 874

Title: Treasure Island

Price(HKD): 126.5

Student: 20210002, CHAN Siu Ming, 98769876

3. Class BkOrderSys [6 marks]

- Create a public Java class **BkOrderSys**, in file **BkOrderSys.java**, as the main class of the book order system (for data records) based on the description below.

ID	Name	Phone Number	Part-Time?
<your student ID>	< Yourself >	55551111	No
20214321	CHOW Betty	88888888	Yes
20215678	AU Candy	12000000	No

Table 3(a): Students

* Phone of AU Candy is supposed to be an invalid number as specified. Do not change it.

Order ID	Book Title	Price	Student
901	"Seven Little Plays"	225.0	CHOW Betty
902	"Treasure Island"	126.5	< Yourself >

Table 3(b): Orders

- Write and complete the following static methods in this program:
 1. **public static StudentInfo addNewFTStudent ()** : generate & return a new **full-time** student
 2. **public static void main(String[] args)** : the program entry point
- The method **addNewFTStudent ()** is used to generate & return a new **full-time** student, by user input.
 - This method accepts no input parameter, and return a new student type **StudentInfo**
 - This method body displays a proper message to ask user to enter one full-time student record as sample output, in the form of: **"<sID> , <sName> , <sPhone>"**
 - This method then creates a new full-time student based on user input, and return the new student.
- The method **main ()** body should complete the following 5 tasks one after one:
 1. Create 3 variables of **StudentInfo**, with the information in Table 3(a) above:
 - Use **your own student** information (**id, name**) for the first as specified in Table 3(a).
 2. Create 2 variables of **BookOrder**, with the information in Table 3(b) above (*Note that you are the student placing an order with Order ID 902*).

3. Display the 2 orders created above, by calling their related method (as sample display below)
4. Create a new student and update (and display) the first book order (i.e., Order ID 901):
 - Ask user to enter and create a new student, by calling method `addNewFTStudent()`
 - Update Order ID 901 with this new student, and display this order again
5. Finally, display your **student's information (name, id, class, year)** before program terminates, similar to the format as the sample output display below:

Sample Program Output

```

--- Book Order System, A2, OOP, 2022 ***
Order ID: 901
Title: Seven Little Plays
Price(HKD): 225.0
Student: 20214321,CHOW Betty,88888888,PartTime

Order ID: 902
Title: Treasure Island
Price(HKD): 126.5
Student: <SID>,<StudentName>,55551111

--- Enter a new Full-Time student in the form <ID,Name,Phone>:
20211234,LAU Andy,98765432
--- Updated Order:
Order ID: 901
Title: Seven Little Plays
Price(HKD): 225.0
Student: 20211234,LAU Andy,98765432

>>> END of main(). Done by <StudentName>, <SID>, <Class>, <Year> <<<
  
```

4. **FOR REFERENCE ONLY:** testing with the given Main Testing File (**MainTA2B.java**)
 - The execution of the given testing main class should produce a similar result as below, if students have properly completed this assignment.

Sample Program Output, when testing with the given Main Testing File

*** START of MAIN TESTING, for A2B ***

** StudentInfo TESTING **

10011001,CHAN Tai Man,3322,PartTime

20022002,CHAN Siu Ming,33333333

** BookOrder TESTING **

Order ID: 20210

Title: ABC

Price(HKD): 100.0

Student: 10011001,CHAN Tai Man,3322,PartTime

*** END of MAIN TESTING ***

5. Evaluation and Submission

- Mark sure you follow the instructions, including the required *naming (e.g. parameters, methods and files)*.
- Compile, Run, Debug, Test and Evaluate your program based on the requirements.
- Submit ALL required files to SOUL:
 - Java Source code files: **StudentInfo.java, BookOrder.java, BkOrderSys.java**
 - Compiled bytecode files: **StudentInfo.class, BookOrder.class, BkOrderSys.class**
- **Do NOT compress/zip** or **rename** the files. Submission work not following instructions and requirements may not be marked or be penalized.
- ***IMPORTANT*** *No Late Submission. Make sure the submitted work is the most updated.*

* Remarks:

- *More testing may be performed during assessment based on the requirements, apart from the given testing cases.*

~ END ~